

- Maven Configuration
 - Do not type the version number of a spring boot dependency in the pom.xml file.
 - If you ctrl+click the spring-boot-starter-parent artifact id in the parent tag, it will take you to the pom file of the spring-boot-starter-parent
 - If you ctrl+click the the artifact id spring-boot-dependencies in the parent tag of the spring-boot-starter-parent pom file, it will take you the spring-boot-dependencies pom file
 - In the spring-boot-dependencies pom file it has all of the listed dependencies and versions that are compatible with each other. If you remove the version property, spring boot will use the versions in this parent file to select the right version
- Application.properties
 - This file can be used to help you set up your database connection from the Java Spring Boot application to the database you're querying
 - For properties you'll need to set each of these properties:
 - Spring.datasource.url - the url where your database is hosted
 - Spring.datasource.username- a user of the database you're trying to access
 - Spring.datasource.password - the password for that user
 - Spring.datasource.driver-class-name - the name of the driver that will be translating java code to communicate with the database you're connecting to
- Lombok
 - A dependency that helps with using annotations that spring uses to create getter and setter methods for a class so a programmer does not have to manually create them
- Annotations that are needed for a Rest Microservice
 - @Repository - sets a class as a "data access object"
 - Means this class handles database storage, retrieval, and data searching
 - @RestController - this communicates to spring that the class with this annotation will handle http request and retrieving and sending data from other HTTP REST services
 - Using this class make its so that you don't have to use annotations like @ResponseBody or @Controller in multiple places in the code, keeping the code cleaner

- **@Component** - this annotation is what gives you the ability to not manually instantiate a class.
 - Spring handles this instantiation of objects behind the scenes for this class and also handles dependency injection(using this in other classes without having to import the class or instantiate any objects)
- **@CrossOrigin**
 - This make annotation helps to ensure that cross - domain requests from applications like react and angular are not blocked due to the browser's same origin policy
- **Java Entities**
 - Using this tag lets spring boot know that class will be a data object that represents a table entry
 - The class fields will represent the data fields for the data table
 - The **@Table** annotation to ensure that it is connected to a specific table
 - **@Id** annotation you use on the field that will present the primary key of the database table
 - **@Column** annotation ties each class field to their respective data field in the table
 - **@GeneratedValue** is an annotation used to create values for your primary key for database entities.
- The **ResponseEntity** class represents the http response that gets sent back to a client. Its properties include:
 - The status code that lets you know if the request is okay, or errored out
 - The header that contains meta-data, such as content type, security tokens, and cache controls
 - The body of the response, which is the actual content
- **JpaSpecificationExecutor** class can be used for sending dynamic queries on the database for more complicated search capabilities
 - You add it to the list of classes you want to extend/inherit in your repository class
 - Spring actually has query methods
 - Examples:
 - **findByFieldNameContaining(String queryName)**
 - The method name gets converted into a sql query that uses the parameter you passed and the field name being referenced in the method to form a condition to append to the WHERE clause
 - However, to combine filters, we would need to hard code each combination of criteria with a query method

- However, the alternative for more complicated queries is using the JpaSpecificationExecutor
 - Create a new class that imports the specification class and handles the criteria building
 - Specification class is a functional interface, which is an interface class with one abstract method
 - When you create a new specification object, you have to pass an implementation for the abstract method that will return a predicate(which is a function that returns a boolean value and is used to test a condition on an object)
 - Note: functional interfaces can have as many static and default methods as it wants as long as it has one abstract method
 - Predicate method will have three parameters
 - Root - represents the entity(class that's supposed to represent structure of data table)
 - root.get(<entity-variable>) is like selecting a field on a table. Can be used to grab a table field for setting a condition
 - Query - specifying the ORDER, DISTINCT, or GROUP BY clauses of a sql query
 - criteriaBuilder - basically the WHERE clause of a sql query
 - criteriaBuilder.equals(<field-name>,<provided-value>) - basically creates a WHERE condition for the sql query that checks if a provided value is equal to the value in the record that it's looking at
 - criteriaBuilder.likes(<field-name>,<provided-value>) - checks if provided value is contained within string value of field
 - Implementation could look like this:
 - Return new Specification<class-entity>() {

@Override Public Predicate

toPredicate(Root<class-entity> root,

Query<class-entity> query,

CriteriaBuilder<class-entity> criteriaBuilder) {

Return

`criteriaBuilder.equals(root.get(<field-name>),<provide-value>);}}` - I would copy this and paste this in notepad and add the necessary spaces to make this look cleaner and make more sense

- Or the implementation could be converted to a lambda expression
 - `(root, query, criteriaBuilder) -> criteriaBuilder.equals(root.get(<field-name>),<provide-value>);` – much cleaner, use this one
 - Functional interface implementation can allow you to convert expressions to lambda expressions
- Should also import the Root, Query, and criteriaBuilder classes for this
- Once it's added to the list of extended classes, you can use a method in the interface called `findAll` that takes in the specification object you provide it
- You can use a `PostMapping` method on the controller so you can send a full data object that will represent the search criteria
 - Pass the search criteria to a `findByCriteria` method that will be implemented by the Dao that will utilize the specification class created
 - For the `dataCriteria` object being passed, set the value of the map to type object. To handle casting, use `instanceof` and assign it to a variable with the type it represents. For example:
 - `Value(object) instanceof <variable-type> variableName`
- In the `findByCriteria` method, you'll be able to initialize a specification object like this: `Specification.unrestricted()`
- After that, you can use a series of if-statements that check if a criteria field is empty, if it's not you can use a `.or(predicate)` or `.and(predicate)` concatenation method to add to the specification object
 - The parameter you're passing to the `.or()` and `.and()` methods are the Specification objects that you return in the Specification methods you created
 - Make sure you re-assign the spec variable the result of calling the `.or()` or `.and()` on the original spec. Just calling

those methods on the spec won't change the object, but will return a new spec object with the or/and with the condition(what the specification object represents) appended.